

HOSTEL MANAGEMENT SYSTEM

Student Table

```
CREATE TABLE Student (  
    student_id INT PRIMARY KEY AUTO_INCREMENT,  
    name VARCHAR(50),  
    usn VARCHAR(20) UNIQUE,  
    gender VARCHAR(10),  
    department VARCHAR(50),  
    year INT,  
    phone VARCHAR(15),  
    address VARCHAR(100)  
);
```

Hostel Table

```
CREATE TABLE Hostel (  
    hostel_id INT PRIMARY KEY AUTO_INCREMENT,  
    hostel_name VARCHAR(50),  
    type VARCHAR(10), -- Boys/Girls  
    total_rooms INT  
);
```

Room Table

```
CREATE TABLE Room (  
    room_id INT PRIMARY KEY AUTO_INCREMENT,  
    hostel_id INT,  
    room_number VARCHAR(10),  
    capacity INT,  
    FOREIGN KEY (hostel_id) REFERENCES Hostel(hostel_id)
```

);

Allocation Table

```
CREATE TABLE Allocation (  
    allocation_id INT PRIMARY KEY AUTO_INCREMENT,  
    student_id INT,  
    room_id INT,  
    join_date DATE,  
    FOREIGN KEY (student_id) REFERENCES Student(student_id),  
    FOREIGN KEY (room_id) REFERENCES Room(room_id)  
);
```

Fee Table

```
CREATE TABLE Fee (  
    fee_id INT PRIMARY KEY AUTO_INCREMENT,  
    student_id INT,  
    amount DECIMAL(10,2),  
    status VARCHAR(20), -- Paid / Pending  
    payment_date DATE,  
    FOREIGN KEY (student_id) REFERENCES Student(student_id)  
);
```

Warden Table

```
CREATE TABLE Warden (  
    warden_id INT PRIMARY KEY AUTO_INCREMENT,  
    name VARCHAR(50),  
    phone VARCHAR(15),  
    hostel_id INT,  
    FOREIGN KEY (hostel_id) REFERENCES Hostel(hostel_id)  
);
```

INSERT QUERIES

Insert Student

```
INSERT INTO Student (name, usn, gender, department, year, phone, address)
VALUES ('Ananya', '1VT21EC001', 'Female', 'ECE', 3, '9876543210', 'Bangalore');
```

Insert Hostel

```
INSERT INTO Hostel (hostel_name, type, total_rooms)
VALUES ('Ganga Hostel', 'Girls', 100);
```

Insert Room

```
INSERT INTO Room (hostel_id, room_number, capacity)
VALUES (1, 'G101', 3);
```

Allocate Room

```
INSERT INTO Allocation (student_id, room_id, join_date)
VALUES (1, 1, '2025-01-01');
```

Fee Payment

```
INSERT INTO Fee (student_id, amount, status, payment_date)
VALUES (1, 45000, 'Paid', '2025-01-10');
```

4. SELECT QUERIES

[View All Students](#)

```
SELECT * FROM Student;
```

Result Grid

Filter Rows:

Edit:

Export/Import:

Wrap Cell Content:

	student_id	name	usn	gender	department	year	phone	address
▶	1	Ananya	1VT21EC001	Female	ECE	3	9876543210	Bangalore
✱	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Result Grid

Form Editor

Field Types

student 1 x

Apply

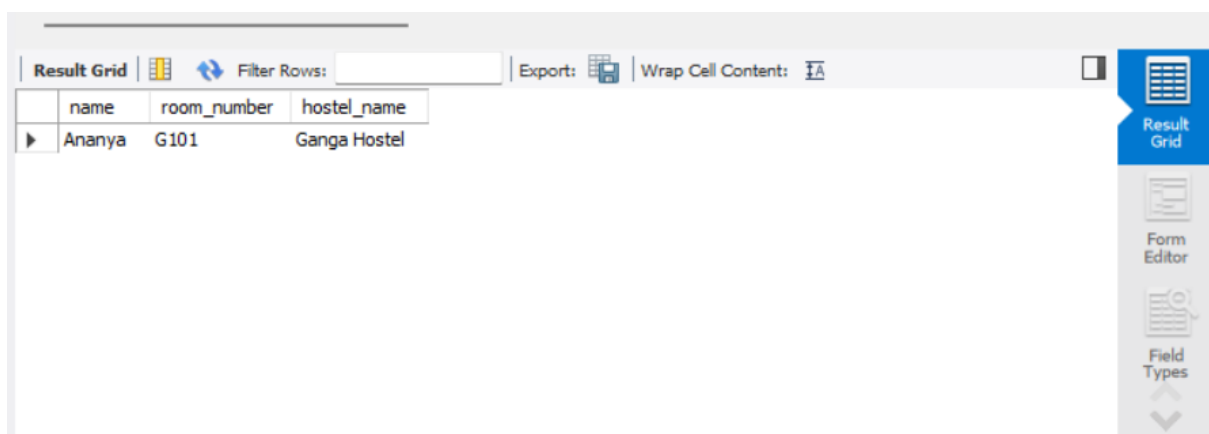
Revert

Students in a Particular Department

```
SELECT name, usn  
FROM Student  
WHERE department = 'ECE';
```

Students Staying in Hostel

```
SELECT s.name, r.room_number, h.hostel_name  
FROM Student s  
JOIN Allocation a ON s.student_id = a.student_id  
JOIN Room r ON a.room_id = r.room_id  
JOIN Hostel h ON r.hostel_id = h.hostel_id;
```



The screenshot shows a database query result grid. The grid has three columns: 'name', 'room_number', and 'hostel_name'. The first row contains the values 'Ananya', 'G101', and 'Ganga Hostel'. The interface includes a toolbar with options like 'Filter Rows', 'Export', and 'Wrap Cell Content'. On the right side, there is a vertical menu with icons for 'Result Grid', 'Form Editor', and 'Field Types'.

name	room_number	hostel_name
Ananya	G101	Ganga Hostel

Students with Pending Fees

```
SELECT s.name, f.amount  
FROM Student s  
JOIN Fee f ON s.student_id = f.student_id  
WHERE f.status = 'Pending';
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

name	amount
------	--------

Result 3 x Read Only

Total Students in Each Hostel

```
SELECT h.hostel_name, COUNT(a.student_id) AS total_students
FROM Hostel h
JOIN Room r ON h.hostel_id = r.hostel_id
JOIN Allocation a ON r.room_id = a.room_id
GROUP BY h.hostel_name;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

hostel_name	total_students
Ganga Hostel	1

Result Grid

Available Rooms

```
SELECT r.room_number, r.capacity
FROM Room r
LEFT JOIN Allocation a ON r.room_id = a.room_id
WHERE a.room_id IS NULL;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

room_number	capacity
-------------	----------

Result Grid

5. UPDATE QUERIES

Update Student Phone Number

```
UPDATE Student
SET phone = '9999999999'
WHERE usn = '1VT21EC001';
```

Update Fee Status

```
UPDATE Fee
SET status = 'Paid', payment_date = CURDATE()
WHERE student_id = 1;
```

6. DELETE QUERIES

Remove a Student

```
DELETE FROM Student
WHERE student_id = 1;
```

Remove Room Allocation

```
DELETE FROM Allocation
WHERE student_id = 1;
```

7. AGGREGATE & FUNCTIONS

Total Fee Collected

```
SELECT SUM(amount) AS total_fee
FROM Fee
WHERE status = 'Paid';
```

Number of Rooms in Each Hostel

```
SELECT hostel_id, COUNT(room_id) AS total_rooms
FROM Room
GROUP BY hostel_id;
```

8. SUBQUERIES

Students Who Have Not Paid Fees

```
SELECT name
FROM Student
WHERE student_id IN (
    SELECT student_id
    FROM Fee
    WHERE status = 'Pending'
);
```

9. VIEWS

```
CREATE VIEW HostelStudents AS
SELECT s.name, h.hostel_name, r.room_number
FROM Student s
JOIN Allocation a ON s.student_id = a.student_id
JOIN Room r ON a.room_id = r.room_id
JOIN Hostel h ON r.hostel_id = h.hostel_id;
```